IBScanUltimate Getting Started Guide

Version 4.3.0 (Jan 23, 2026)

Copyright @2011-2024, Integrated Biometrics LLC. All Rights Reserved

INTEGRATED BIOMETRICS

Table of Contents

REVISION HISTORY 3

1. SDK CONTENTS 4

1.1. Packages 4

1.2. Scanner Model Support 4

1.3. Further Information 4

2. INSTALLATION GUIDES 6

2.1. Windows Installation Guide 6

Installing the SDK 6

SDK Contents 6

Sample Applications 6

2.2. Linux (Intel) Installation Guide 8

Opening the SDK 8

SDK Contents 8

Installing the IBScanUltimate Library 8

Compiling and Running the Sample Application 9

2.3. Linux (ARM) Installation Guide 11

Opening the SDK 11

SDK Contents 11

Dependencies 12

Compiling the Sample Application 12

Installing the IBScanUltimate Library 13

2.4. Android Installation Guide 15

SDK Contents 15

Setting up an Android Studio with IBScanUltimate 16

Targeting Android API Level on Android App (Using Columbo Mini) 17

Opening and Running the Example Project 18

The IBScanUltimate Java Application Interface 19

The IBScanUltimate C/C++ Application Interface 20

Using USB Devices on Android 20

3. FINGERPRINT CAPTURE GUIDE 23

3.1. Flat Print Captures 23

3.2. Rolled Print Captures **24**

3.3. Mannix product Capture guide **26**

3.3.1. Rolled Print capture 26

3.3.2. Lower Palm Capture 27

3.3.3. Upper Palm Capture 28

3.3.4. Writer's Palm Capture 29

3.4. Spoof usage **30**

4. ISO TEMPLATE SUPPORT **31**

4.1. Introduce ISO Template **31**

4.2. Fingerprint image Export to ISO Template **31**

4.3. Verify ISO template with BiOCTS **32**

Revision History

Date	Author	Remarks
2024/02	Milton	Added Mannix palm capture guide
2023/03	Milton	Removed EOL(End-Of-Line) samples description
2022/05	Milton	Added ISO Template support
2020/01	Milton	Added spoof usage
2013/10	BAN	Corrected typographical error in Linux (ARM) getting started section.

1. SDK Contents

1.1. Packages

The full IBScanUltimate SDK consists of six separate packages for individual operating systems. The core of each package is the IBScanUltimate library (distributed as a DLL on Windows and shared object libraries on other platforms) that provides an API for capturing images from IB scanners. One or more sample applications in each package demonstrate the use of the library. A Java interface is also available, through a JNI bridge (distributed as a DLL on Windows) and a Java-language wrapper (distributed as a JAR).

The following are the provided packages, where “x.y.z” is the version number such as “3.4.4”:

Platform	Architecture	Compatible OS	Sample	Package File Name	Requirement
Windows	32bit	Windows7, Windows10, Windows 11	C/C++, C#, VB.net, Java	IBScanUltimateSDK Setup x.y.z.exe	
	64bit			IBScanUltimateSDK(x64) Setup x.y.z.exe	
Linux	32bit	Most of Linux Distributions	C/C++, Java	IBScanUltimate_x86_x.y.z.tgz	Requires Linux Kernel version is 2.6 or higher
	64bit			IBScanUltimate_x64_x.y.z.tgz	
	armv7	Most of Embedded Linux Distributions		IBScanUltimate_armv7a_x.y.z.tgz	
	armv8			IBScanUltimate_armv8a_x.y.z.tgz	
Android	arm64-v8a	Google Android OS	Java based Android sample	IBScanUltimate_Android_x.y.z.tar.gz	Requires Android version is 5.x or higher
	armeabi-v7a				
	x86				
	x86_64				

1.2. Scanner Model Support

The latest Integrated Biometrics fingerprint devices supported by the IBScanUltimate SDK are listed below.

[Product List Link of Integrated Biometrics]

https://integratedbiometrics.com/products

1.3. Further Information

For more information about the C/C++ API of the IBScanUltimate library, please refer to *IBScanUltimate API Manual For C.pdf*.

For more information about the Java API (including the Android API) of the IBScanUltimate library, please refer to *IBScanUltimate API Manual for Java (and Android).pdf*.

For the version history of IBScanUltimate, please refer to *IBScanUltimate Version History.pdf*.

For more information about a scanner model, please refer to its hardware manual.

2. Installation Guides

Each of the following sections step through the installation procedure on one platform and summarizes the contents that will be installed with the SDK.

2.1. Windows Installation Guide

Installing the SDK

Execute (usually by double-clicking) the Windows installer, which will be

IBScanUltimate SDK Setup x.y.z.exe

for the 32-bit content and

IBScanUltimate(x64) SDK Setup.x.y.z.exe

for the 64-bit content. Progress through the automatic installer; we recommend accepting default values whenever prompted.

SDK Contents

The SDK contains the library and sample application needed to start developing a Windows application that interfaces with an IB scanner. The material is separated into several directories, including the following:

- The */Bin* directory contains compiled sample applications, the IBScanUltimate DLL (*IBScanUltimate.dll*), and the DLLs and JARs for the Java interface (*IBScanCommon.jar*, *IBScanUltimate.jar*, *IBScanUltimateJNI.dll*).
- The */Driver* directory contains the drivers for IB scanners. These are installed with the SDK installer.
- The */Include* directory contains the include files for the C interface of IBScanUltimate.
- The */Lib* directory contains the compiled IBScanUltimate library file for linking.
- The */Sample sources* directory contains the source for the sample applications. The applications are separated by language.

Sample Applications

The compiled sample applications should appear within the program menu, which will also link to the folder containing the source for the applications. The following applications are provided (listed by the names that appear in the program menu):

- A series of basic samples for several different languages, each offering the same essential GUI and controls for basic scanner operations:
  - IBScanUltimate_SampleForC# is the basic sample for C#.
  - IBScanUltimate_SampleForVBNet is the basic sample for VB.Net.

  - IBScanUltimate_SampleForVC is the basic sample for C/C++.
- IBSU_FunctionTester (a sample for C/C++) enumerates the API functions presented by IBScanUltimate to allow for testing of individual library features and presentation of results.

2.2. Linux (Intel) Installation Guide

Opening the SDK

Please copy the file *IBScanUltimate_x86_x.y.z.tgz* or *IBScanUltimate_x64_x.y.z.tgz* to your development system. Extract the contents with the following command (substituting `x64` for `x86`, if necessary, and the version number for `x.y.z`):

```
# tar zxvfp IBScanUltimate_x86_x.y.z.tgz
```

```
ibkr@ubuntu: ~
ibkr@ubuntu:~$ tar xvfzp IBScanUltimate_x64_3.9.4.tgz
IBScanUltimate_x64_3.9.4/
IBScanUltimate_x64_3.9.4/include/
IBScanUltimate_x64_3.9.4/include/IBScanUltimate.h
IBScanUltimate_x64_3.9.4/include/IBScanUltimateApi.h
IBScanUltimate_x64_3.9.4/include/IBScanUltimateApi_err.h
IBScanUltimate_x64_3.9.4/include/IBScanUltimateApi_defs.h
IBScanUltimate_x64_3.9.4/include/LinuxPort.h
IBScanUltimate_x64_3.9.4/include/ReservedApi.h
IBScanUltimate_x64_3.9.4/lib/
IBScanUltimate_x64_3.9.4/lib/libAKXUS.so.2
IBScanUltimate_x64_3.9.4/lib/libuvc.so.0.0.6
IBScanUltimate_x64_3.9.4/lib/libAKXUS.so
IBScanUltimate_x64_3.9.4/lib/libuvc.so
IBScanUltimate_x64_3.9.4/lib/libIBScanUltimate.so
IBScanUltimate_x64_3.9.4/lib/IBScanCommon.jar
IBScanUltimate_x64_3.9.4/lib/libIBScanUltimateJNI.so
IBScanUltimate_x64_3.9.4/lib/IBScanUltimate.jar
IBScanUltimate_x64_3.9.4/lib/libuvc.so.0
IBScanUltimate_x64_3.9.4/lib/libAKXUS.so.2.0.10
IBScanUltimate_x64_3.9.4/install/
IBScanUltimate_x64_3.9.4/install/install-IBScanUltimate_armv7_gnueabihf.sh
IBScanUltimate_x64_3.9.4/install/install-IBScanUltimate_armv8.sh
IBScanUltimate_x64_3.9.4/install/install-IBScanUltimate.sh
IBScanUltimate_x64_3.9.4/install/install-IBScanUltimate_armv7_gnueabi.sh
```

SDK Contents

The SDK contains the library and sample application needed to start developing a Linux application that interfaces with an IB scanner. The material is separated into three directories:

The */include* directory contains the include files for the C interface of IBScanUltimate.

The */install* directory contains a script (*Install-IBScanUltimate*) to install the IBScanUltimate library as well as the library itself (*libIBScanUltimate.so*).

The */Samples* directory contains source code for the sample applications.

Installing the IBScanUltimate Library

First, plug in the IB USB device to your USB host port.

Second, install the IB driver library on your Linux system with these commands (substituting `x64` for `x86`, if necessary, and the version number for `x.y.z`):

```
# cd IBScanUltimate_x86_x.y.z/install
# sudo ./install-IBScanUltimate
```

Some Linux distributions (like Ubuntu) need root access, obtained with the `sudo` command, to install the driver library.

```
ibkr@ubuntu: ~/IBScanUltimate_x64_3.9.4/install
ibkr@ubuntu:~$ cd IBScanUltimate_x64_3.9.4/install/
ibkr@ubuntu:~/IBScanUltimate_x64_3.9.4/install$ sudo bash install-IBScanUltimate
.sh
[sudo] password for ibkr:
Copied /home/ibkr/IBScanUltimate_x64_3.9.4/install/../lib/libIBScanUltimate.so t
o /usr/lib
Copied /home/ibkr/IBScanUltimate_x64_3.9.4/install/../lib/libIBScanUltimateJNI.s
o to /usr/lib
PAD Library not exist, Skipping..
Copied /home/ibkr/IBScanUltimate_x64_3.9.4/install/../lib/libAKXUS.so.2.0.10 to
/usr/lib
Copied /home/ibkr/IBScanUltimate_x64_3.9.4/install/../lib/libAKXUS.so.2 to /usr/
lib
Copied /home/ibkr/IBScanUltimate_x64_3.9.4/install/../lib/libAKXUS.so to /usr/li
b
Copied /home/ibkr/IBScanUltimate_x64_3.9.4/install/../lib/libuvc.so.0.0.6 to /us
r/lib
Copied /home/ibkr/IBScanUltimate_x64_3.9.4/install/../lib/libuvc.so.0 to /usr/li
b
Copied /home/ibkr/IBScanUltimate_x64_3.9.4/install/../lib/libuvc.so to /usr/lib
ibkr@ubuntu:~/IBScanUltimate_x64_3.9.4/install$
```

Compiling and Running the Sample Application

Now you can compile and run our sample program:

```
# cd ../Samples/Linux/TestScan
# make
# sudo ./testScanU
```

Depending on the permissions you have granted to the user for accessing USB devices, you may need to grant root access to run the sample program.

```
ibkr@ubuntu: ~/IBScanUltimate_x64_3.9.4/Samples/Linux/TestScan
ibkr@ubuntu:~/IBScanUltimate_x64_3.9.4/Samples/Linux/TestScan$ sudo ./bin/testScanU
IBScanUltimate Product version: 3.9.4.0, File version: 3.9.4.0
Found 1 devices attached
FIVE-0_1.0.5 S/N(FV11100-SAM00013-000K) on USB
Ready.   Enter choice:
        1. Start capture for flat single finger.
        2. Start capture for flat two fingers.
        3. Start capture for rolling single finger.
        4. Start capture for flat four fingers.
        5. Abort Capture
Files   6. Options
        7. Set Preview Image
        8. Set Spoof function control
        9. End program
:==>1

Initializing device... 0%

Initializing device... 30%

Initializing device... 70%

Initializing device... 90%

Initializing device... 100%
 Setting up for scan with callback...Displayed 'C'=Image callback.

Ready.   Enter choice:
        1. Start capture for flat single finger.
        2. Start capture for flat two fingers.
        3. Start capture for rolling single finger.
        4. Start capture for flat four fingers.
        5. Abort Capture
        6. Options
        7. Set Preview Image
        8. Set Spoof function control
        9. End program
:
-- Finger count changed -- Device= 0, State= NON-FINGER

-- Finger quality changed -- Device= 0, Qualities= NOT_PRESENT, NOT_PRESENT, NOT_PRESE
NT, NOT_PRESENT
CCCCCCCCCCCCCCCCCCC
-- Finger count changed -- Device= 0, State= FINGER_COUNT_OK

-- Finger quality changed -- Device= 0, Qualities= NOT_PRESENT, NOT_PRESENT, NOT_PRESE
NT, POOR
CCCCC
-- Finger quality changed -- Device= 0, Qualities= NOT_PRESENT, NOT_PRESENT, NOT_PRESE
NT, FAIR
CCCCCCCC
-- Finger quality changed -- Device= 0, Qualities= NOT_PRESENT, NOT_PRESENT, NOT_PRESE
NT, GOOD

CompleteAcquisition

-- Finger quality changed -- Device= 0, Qualities= NOT_PRESENT, NOT_PRESENT, NOT_PRESE
NT, GOOD

Stopped. 5.8 frames per second

Flat single finger Image acquisition complete


Press enter!
```

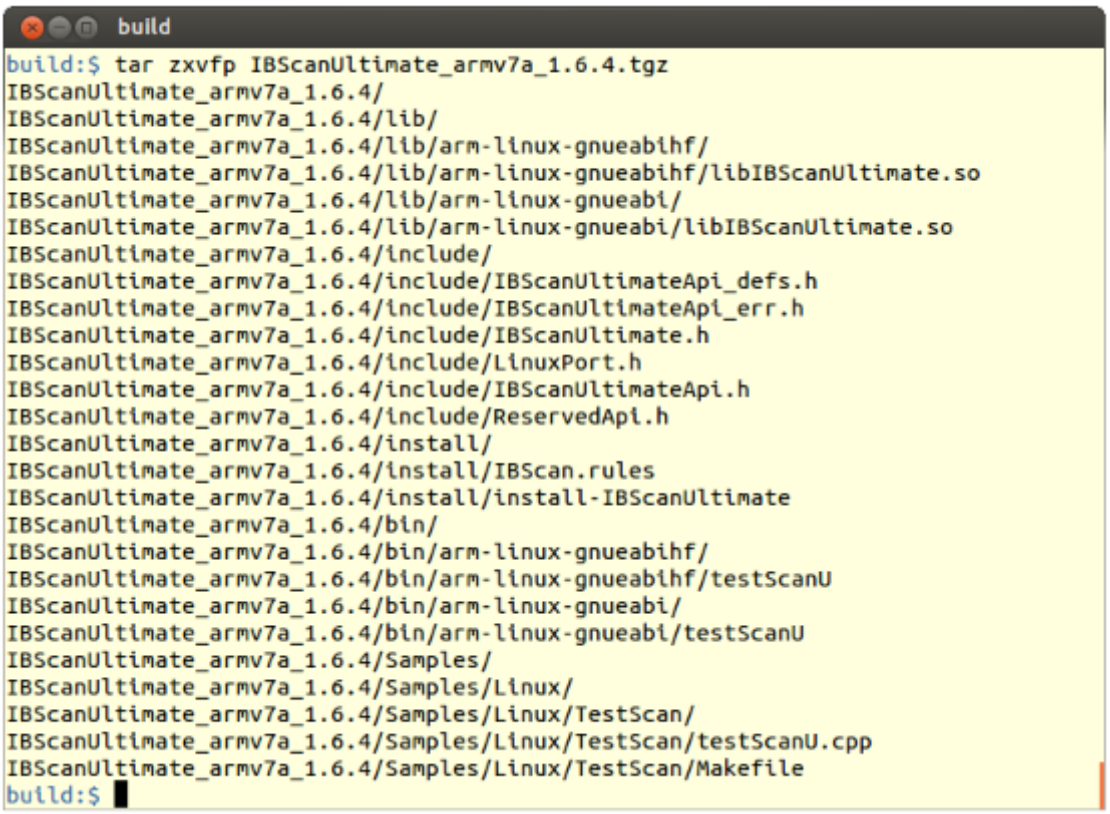
2.3. Linux (ARM) Installation Guide

Opening the SDK

Copy the file *IBScanUltimate_armv7a_x.y.z.tgz* to either your host (your desktop Linux computer or VM) or target (the ARM Linux device). Since the sample application may need to be rebuilt, the choice will depend on the location of your build tools (either a cross-compiler on the host or native tools on the target). (In the sample commands below, the build and target systems are differentiated by their prompts—`Build:$` versus `Target:$`—even though these will coincide if you build natively.)

Extract the contents with the following command (substituting the version number for `x.y.z`):

```
Build:$ tar zxvfp IBScanUltimate_armv7a_x.y.z.tgz
```



```
build
build:$ tar zxvfp IBScanUltimate_armv7a_1.6.4.tgz
IBScanUltimate_armv7a_1.6.4/
IBScanUltimate_armv7a_1.6.4/lib/
IBScanUltimate_armv7a_1.6.4/lib/arm-linux-gnueabihf/
IBScanUltimate_armv7a_1.6.4/lib/arm-linux-gnueabihf/libIBScanUltimate.so
IBScanUltimate_armv7a_1.6.4/lib/arm-linux-gnueabi/
IBScanUltimate_armv7a_1.6.4/lib/arm-linux-gnueabi/libIBScanUltimate.so
IBScanUltimate_armv7a_1.6.4/include/
IBScanUltimate_armv7a_1.6.4/include/IBScanUltimateApi_defs.h
IBScanUltimate_armv7a_1.6.4/include/IBScanUltimateApi_err.h
IBScanUltimate_armv7a_1.6.4/include/IBScanUltimate.h
IBScanUltimate_armv7a_1.6.4/include/LinuxPort.h
IBScanUltimate_armv7a_1.6.4/include/IBScanUltimateApi.h
IBScanUltimate_armv7a_1.6.4/include/ReservedApi.h
IBScanUltimate_armv7a_1.6.4/install/
IBScanUltimate_armv7a_1.6.4/install/IBScan.rules
IBScanUltimate_armv7a_1.6.4/install/install-IBScanUltimate
IBScanUltimate_armv7a_1.6.4/bin/
IBScanUltimate_armv7a_1.6.4/bin/arm-linux-gnueabihf/
IBScanUltimate_armv7a_1.6.4/bin/arm-linux-gnueabihf/testScanU
IBScanUltimate_armv7a_1.6.4/bin/arm-linux-gnueabi/
IBScanUltimate_armv7a_1.6.4/bin/arm-linux-gnueabi/testScanU
IBScanUltimate_armv7a_1.6.4/Samples/
IBScanUltimate_armv7a_1.6.4/Samples/Linux/
IBScanUltimate_armv7a_1.6.4/Samples/Linux/TestScan/
IBScanUltimate_armv7a_1.6.4/Samples/Linux/TestScan/testScanU.cpp
IBScanUltimate_armv7a_1.6.4/Samples/Linux/TestScan/Makefile
build:$
```

SDK Contents

The SDK contains the library and sample application needed to start developing a Linux application that interfaces with an IB scanner. The material is separated into five directories:

> The */bin* directory contains versions of the compiled sample application (*testScanU*), separated into directories by ABI. These binaries have been compiled dynamically against the dependent libraries under a cross-compiler and may not execute on all targets. You may need to recompile the application

with the proper toolchain.

The */include* directory contains the include files for the C interface of IBScanUltimate.

The */install* directory contains a script (*Install-IBScanUltimate*) to install the IBScanUltimate library.

The */lib* directory contains versions of the IBScanUltimate library (*libIBScanUltimate.so*) separated into directories by ABI.

The */Samples* directory contains source code for the sample application.

The distribution has separate binaries and libraries for two ABIs:

The arm-linux-gnueabi version should be installed on platforms with system libraries and applications built for the "soft" floating-point ABI, equivalent to the GCC flags

```
-mthumb -mfloat-abi=softfp -march=armv7-a
```

The arm-linux-gnueabihf version should be installed on platforms with system libraries and applications built for the "hard" floating-point ABI, equivalent to the GCC flags

```
-mthumb -mfloat-abi=hard -march=armv7-a
```

Dependencies

The IBScanUltimate library requires that libusb 1.0-dev is installed on the target. On Ubuntu, you would use `apt-get`:

```
Target:$ sudo apt-get install libusb-1.0-0-dev
```

When compiling your application, you must dynamically link with both of these libraries, *libIBScanUltimate.so*, and several standard C++ libraries:

```
Build:$ gcc myapp.c -o myapp -l IBScanUltimate -l stdc++ -
l pthread
```

This command ignores the inclusion of the IBScanUltimate includes and library directories.

Depending on the file installed by the package manager, the same links may be needed on the target system to run an application gathering fingerprints with IBScanUltimate.

Compiling the Sample Application

On some targets, the provided sample binaries may not execute because these are compiled against recent versions of the C runtime. To recompile the sample, navigate to the *Samples/Linux/TestScan* directory. Set and export the variable `CROSS_COMPILE` to the prefix of your tools, if you are not building natively, then set and export the

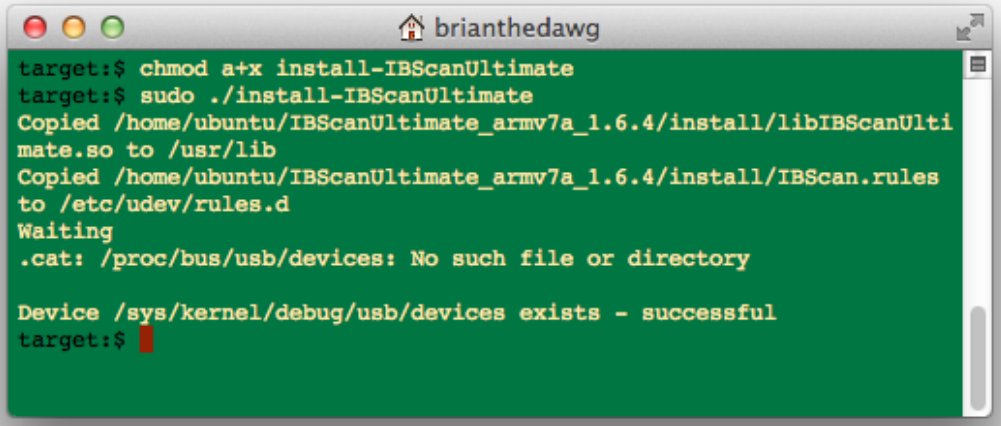
variable `ARCHABI` to either `arm-linux-gnueabi` or `arm-linux-gnueabi`, depending on the ABI. Finally, make the application (*testScanU*), which will be output in a local *bin* directory.

```
Build:$ cd
IBScanUltimate_armv7a_x.x.x/Samples/Linux/TestScan
Build:$ export CROSS_COMPILE=/mygcc/bin/arm-linux-gnueabi-
Build:$ export ARCHABI=arm-linux-gnueabi
Build:$ make
```

If you are cross-compiling, native versions of libusb-1.0 must be “installed” in your toolchain’s runtime library structure or on a library search path.

Installing the IBScanUltimate Library

A script is provided to install the libIBScanUltimate library.
Execute this script (sudo sh *install-IBScanUltimate*), found in the *install* directory of the distribution, to your target system, with the appropriate version of *libIBScanUltimate.so*, and (optionally) the compiled sample application.



Once the files are located on your target, execute the script by your platform.
Request root access if necessary.

```
[Linux armv7-gnueabihf]
Target:$ sudo sh install-IBScanUltimate_armv7_gnueabihf.sh

[Linux armv7-gnueabihf]
Target:$ sudo sh install-IBScanUltimate_armv7_gnueabi.sh

[Linux armv8]
Target:$ sudo sh install-IBScanUltimate.sh
```

```
target:$ sudo ./testScanU
[sudo] password for ubuntu:
IBScanUltimate Product version: 1.6.3.0, File version: 1.6.3.0
Found 1 devices attached
WATSON MINI_1.0.2 S/N(1234-00075) on USB

Ready.   Enter choice:
        1. Start capture for flat single finger.
        2. Start capture for flat two fingers.
        3. Start capture for rolling single finger.
        4. Abort Capture
        5. End program
:==>1

Initializing device... 0%

Initializing device... 100%
 Setting up for scan with callback...Displayed 'C'=Image callback.

Ready.   Enter choice:
        1. Start capture for flat single finger.
        2. Start capture for flat two fingers.
        3. Start capture for rolling single finger.
        4. Abort Capture
        5. End program
:
-- Finger count changed -- Device= 0, State= NON-FINGER
CCCCCCC
-- Finger count changed -- Device= 0, State= FINGER_COUNT_OK
CCCCCCCCCCCCCCCC
Stopped. 4.6 frames per second

Flat single finger Image acquisition complete
Saving image...
NFIQ score is 2


Press enter!
```

Run the sample application

Now you can run the sample application:

```
Target:$ sudo ./testScanU
```

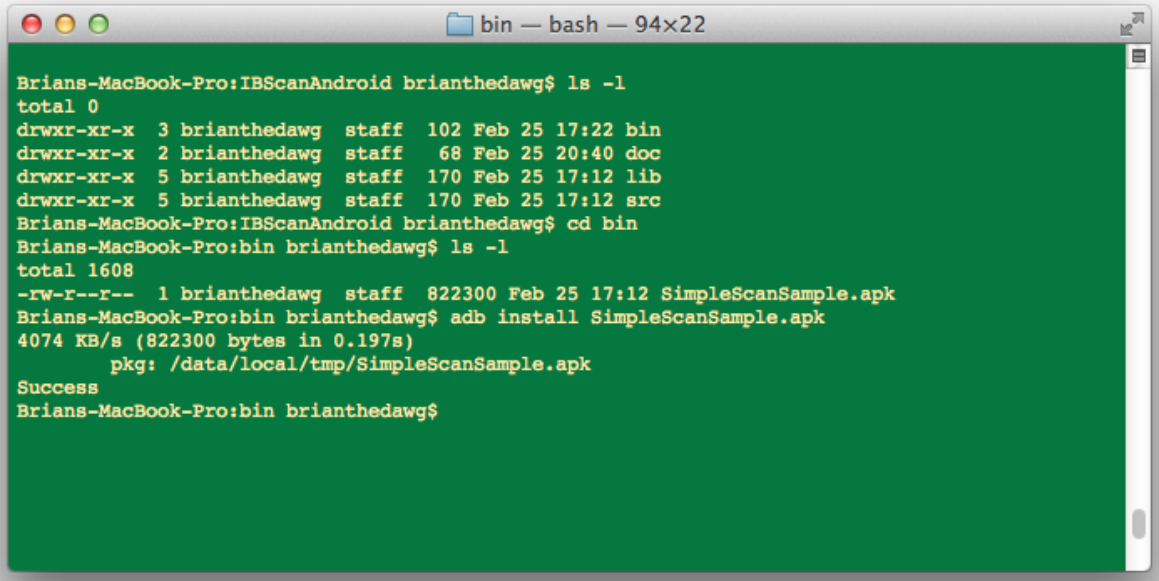
Depending on the user's permissions for accessing USB devices, you may need to grant root access to run the sample program.

2.4. Android Installation Guide

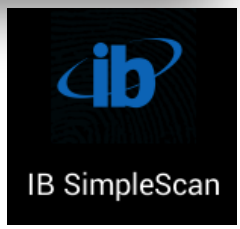
SDK Contents

The SDK contains the applications and libraries needed to start developing an Android application that interfaces with an IB scanner. The material is separated into five directories:

- The */bin* directory contains only one file: a compiled version of the IB SimpleScan application. To install this application, open a terminal window, move to that directory and use ADB to install the application on your connected Android phone or tablet. Make sure that *adb* (or *adb.exe*, on Windows), located in the Android SDK's *platform-tools* directory, is on the PATH or referenced by its full path.



If you have more than one Android device connected or have an emulator open, you may need to use the ADB –s switch with the appropriate identifier to direct the installation to your desired target. You will find the installed app listed under the name "IB SimpleScan".



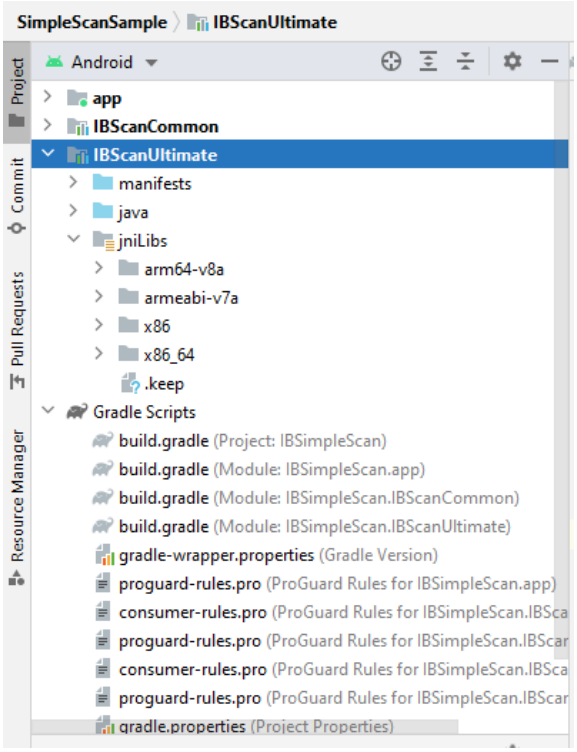
- The */lib* directory contains four folders, each containing compiled versions of the libraries you will need for development.
- The */include* directory contains the include files for the native interface of IBScanUltimate.
- The */Samples* directory contains source code for the SimpleScan sample application, the libusb distribution specifically for unrooted Android systems, and the wrapper which exposes to app a convenient Java interface for communicating with IB scanners.

Setting up an Android Studio with IBScanUltimate

The Android project structure is shown on the right. The IBScanCommon and IBScanUltimate project exists here.

IBScanCommon project is only configured by Java language.

IBScanUltimate project configured by Java + Native Library combined.
In the IBScanUltimate project, this contains the jniLibs folder and this folder includes C based native library files included by ABI (arm64-v8a, armeabi-v7a,x86,x86_64)



You need to import these two Modules(IBScanCommon+IBScanUltimate) on your Android studio project to build your Android app.

If you prefer to import the library file instead of Module structure, you can use AAR(Android Archive) files.
In our Release package /Lib directory, you can find two *.aar files.

- ▷ iBScanCommon-release.aar
- ▷ iBScanUltimate-release.aar

In iBScanUltimate-release.aar file, it includes the following native library files.

- ▷ libusb.so
- ▷ libuvc.so
- ▷ libAKXUS.so
- ▷ libIBScanUltimate.so
- ▷ libIBScanUltimateJNI.so
- ▷ libLiveFinger2.so
  (PAD enabled package)

Targeting Android API Level on Android App (Using Columbo Mini)

If you plan to use IB's Columbo Mini device on an Android platform,
the Columbo Mini works as a Camera(UVC Webcam).

Recently Google increased/enforced Security on Android platform about Camera and USB, So If you use Columbo Mini on latest Android device, You need to check your app's Android API level.

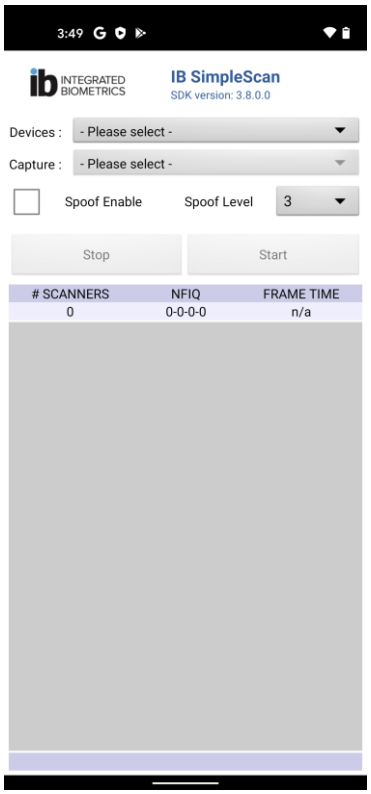
If the Android app's Target API level is higher(>=) than 28(Android P)
→ You need to add **Camera** permission on your Android Manifest.xml file
for recognizing the Columbo Mini in the Android App

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.integratedbiometrics.ibsimplescan">

    <uses-feature android:name="android.hardware.usb.host"/>
    <uses-permission android:name="android.permission.CAMERA"/>

    <application>
        …
    </application>
</manifest>
```

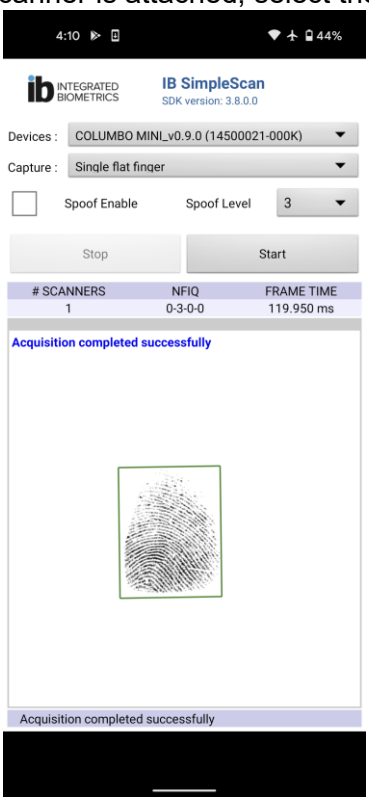
Opening and Running the Example Project



The IB SimpleScan example project is located in the */Sample/Android/SimpleScanSample* directory. To open this project in Android Studio

- Click “Projects” → “Open”

Once Project is loaded, the sample will appear under the app list with the name “IB SimpleScan”. When the app is opened, it will detect any attached scanners or wait until it receives a notification that a scanner has been attached. After an IB scanner is attached, select the scan type and press the “Start” button to initialize the scanner then capture a scan. After the scan is completed, holding on the image view with the fingerprint will reveal a pop-up menu with options to e-mail the image and show an enlarged version.



The IBScanUltimate Java Application Interface

The SDK libraries provide a Java interface for the convenience of the application, all within the package `com.integratedbiometrics.ibscanultimate`. The five principal classes of this package exposes the functionality of the native IBScanUltimate libraries on Linux, with a more optimal interface for Java programming. For more information beyond the brief introduction below, please see the Java API manual.

- `IBScan`: The scanner manager. The primary class, `IBScan`, manages scanners. Typically, an app's main Activity will get a handle to the single instance of the IBScan class in its `onCreate()` method:

  ```
  this.m_ibScan =
      IBScan.getInstance(this.getApplicationContext());
  ```

  With this handle, the application can use the services IBScan provides, including getting a description the SDK version, getting the description of a device, and opening a handle to a scanner.

- `IBScanListener`: The scanner manager event listener. Typically, the application will register an IBScanListener to receive notifications:

  ```
  this.m_ibScan.setScanListener(this);
  ```

  In this case, the Activity implements the IBScanListener interface itself, e.g.,

  ```
  public class MyActivity extends Activity
     implements IBScanListener {
  ```

  The listener must override a number of methods. The listener (covered more in the section on USB devices below) alerts the app about events important to the cycle of device attachment (attached, permission granted, device count changed, detached). The other two are important to the procedure of opening a device.

- `IBScanDevice`: The scanner handle. A handle to a scanner is returned from one of the synchronous `IBScan openDevice()` functions, or through the `scanDeviceOpenComplete()` callback when using one of the asynchronous `openDeviceAsynch()` functions.

- `IBScanDeviceListener`: The scanner event listener. Typically, the application will register an IBScanDeviceListener to receive notifications:

  ```
  this.m_ibScanDevice.setScanDeviceListener(this);
  ```

  In this case, the Activity implements the IBScanDeviceListener interface itself, e.g.,

  ```
  public class MyActivity extends Activity implements
                                  IBScanDeviceListener {
  ```

  The listener must override a number of methods. All of these are called during capture (after calling `beginCaptureImage()`) to allow the app to respond dynamically by, for example, updating its display of the current scan without needing to poll the device.

- IBScanException: The scanner exception. Most `IBScan` and `IBScanDevice` returns may throw an `IBScanException`, which the app must catch. An

exception may warn that an operation is illegal (such as setting a read-only property) or that an error occurred (such as a I/O error from a communication failure).

```
int deviceIndex = 0;
try
{
    IBScanDevice myDevice = this.m_ibScan.open(deviceIndex);
}
catch (IBScanException ibse)
{
    System.out.println("Failed to open with exception " +
        ibse.getCode().toString();
}
```

The IBScanUltimate C/C++ Application Interface

In general, the Java API will be the best choice for an application wishing to control an IB scanner, since it takes care of the messy details of wrapping the native IBScanUltimate libraries. Since some applications may prefer to code certain operations (for example, image processing) in C or C++, the native library interface is accessible. Conveniently, this Android C/C++ API is identical to the Linux API, and can be used in the same way (except for peculiarities in using USB devices on Android; see the next section). Please see the Linux API manual for more information on that API.

Using USB Devices on Android

Android-based systems are locked-down; an app can access little beyond the boundaries of its process and resources. Permissions to access some features, such as external storage, are advertised by the app in its manifest, and the user is warned when installing the app. For USB devices, permission is granted on a device-by-device basis. An app can request permission to access a certain device programmatically (or become a default device by registering interest in certain categories by vendor and product ID or class), whereupon the user is prompted to approve or deny in a popup dialog. If the user approves, the app can access the device.

As part of this `IBScanListener` interface, several methods must be overridden; for the purposes of this discussion, two are most relevant. When a IB scan device is attached, the `scanDeviceAttached()` method will be called, offering the app an opportunity to request permission to access it:

```
@Override
public void scanDeviceAttached(int deviceId)
```

```
{
    /* Check whether we have permission to access this device.
    * Request permission so it will appear as an IB scanner. */
    boolean hasPermission =
        this.m_ibScan.hasPermission(deviceId);
    if (!hasPermission)
    {
       this.m_ibScan.requestPermission(deviceId);
    }
}
```

After the application responds to a request, another listener method (`scanDevicePermisionGranted()`) will be invoked; if permission is granted, IBScanUltimate will refresh the list of devices it maintains internally then notify the application that the number of connected devices has changed with `scanDeviceCountChanged()` so the app will realize more scanners are available. (Of course, this method will also be invoked if a scanner is detached, for the opposite reason.)

```
@Override
public void scanDeviceCountChanged(int deviceCount)
{
    /* The number of recognized accessible scanners has changed.
     * Let's refresh the list of scanners we can choose from. */

    .
    .
    .
}
```

If the app enters a state when Android's `UsbManager` is not notifying it of device attachments, the app may need to manually iterate through the manager's list of devices and request permission to access any inaccessible scanners:

```
/* Make sure there are no USB devices attached that are IB
 * scanners for which permission has not been granted.  For any
 * that are found, request permission; we should receive a
 * callback when permission is granted or denied and then when
 * IBScan recognizes that new devices are connected. */
UsbManager manager = (UsbManager)this.getApplicationContext()
    .getSystemService(Context.USB_SERVICE);
HashMap<String, UsbDevice> deviceList = manager.getDeviceList();
Iterator<UsbDevice> deviceIterator =
    deviceList.values().iterator();
```

```
while (deviceIterator.hasNext())
{
    UsbDevice device      = deviceIterator.next();
    boolean   isScanDevice = IBScan.isScanDevice(device);

    if (isScanDevice)
    {
        boolean hasPermission = manager.hasPermission(device);
        if (!hasPermission)
            this.m_ibScan.requestPermission(device.getDeviceId());
    }
}
```

3. Fingerprint Capture Guide

3.1. Flat Print Captures

The finger should be both properly aligned and maintain contact with the scanner's case. As shown in Figure 1a, on a Watson Mini, the finger should oppose the USB cable and be perpendicular to the side. From Figure 1b, you can see that the finger contacts the plastic case, as necessary on a Watson Mini, Columbo, and Sherlock sensors; on a Watson scanner, contact must be made with the metal strip across the scanners bottom.

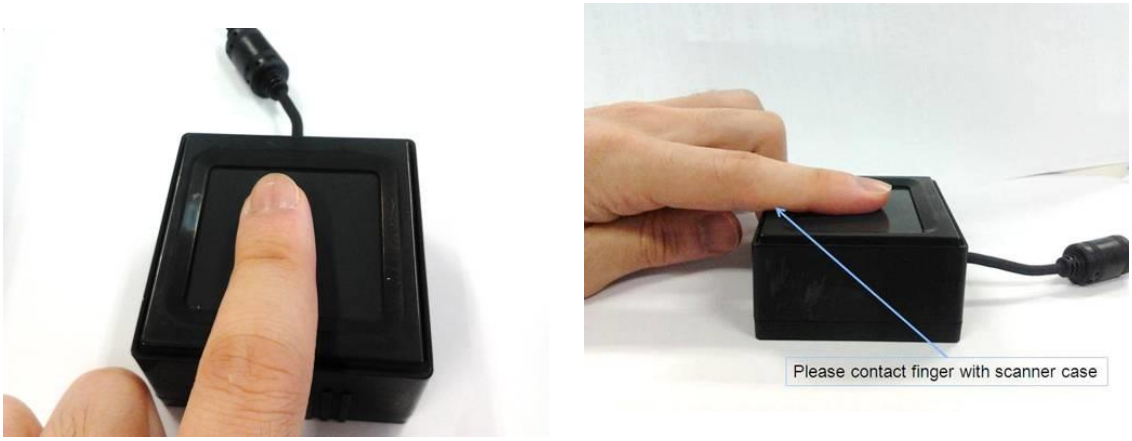


Figure 1. Good finger placement
(a, left) proper finger alignment; (b, right) finger contact with scanner case

Figure 2. Bad finger placement
(a, left) improper alignment, should be opposite USB cable; (b, middle) improper alignment, should be opposite USB cable; (c, right) no contact with scanner case

3.2. Rolled Print Captures

The recommendations for flat captures apply to roll captures, too. Both proper fingerprint alignment and contact with the scanner ground are necessary. The rolled capture is a sequence of three steps. As shown in Figure 3, the finger should be kept on the sensor surface until the flat print capture completes; in our sample programs, this condition is indicated with a red line (the line that tracks the capture location) in the fingerprint display.

Figure 3. Rolled capture, step 1. The finger should be kept on the sensor surface until the flat print capture completes

Then, as shown in Figure 4, the finger should be rolled left until the entire left side is captured; in our sample programs, this condition is indicated when the line that tracks the capture turns from red to green.

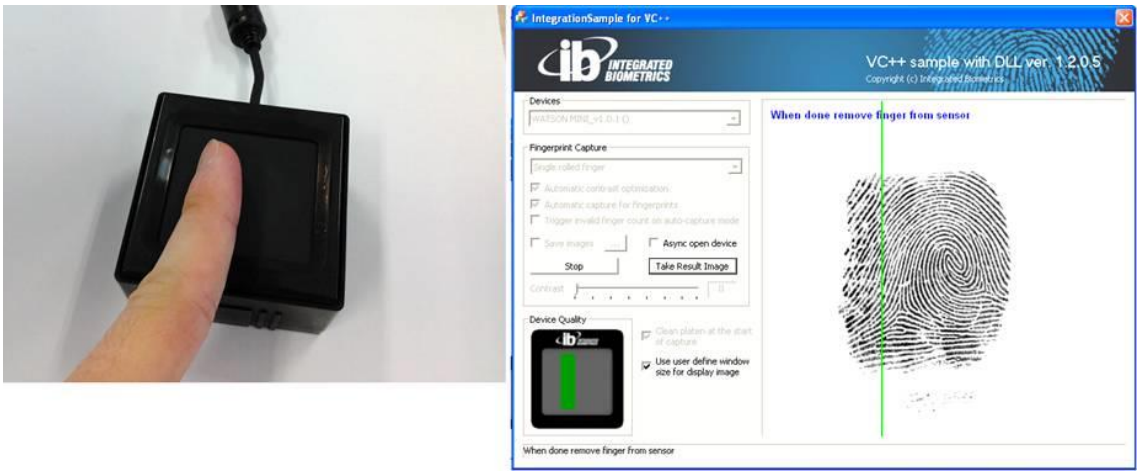


Figure 4. Rolled capture, step 2. The finger should be rolled left until the entire left side is captured

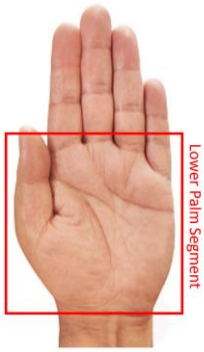
Finally, as shown in Figure 5, roll the finger back to the right until the entire right side is captured. The rolled capture completes when the finger is removed from the platen.

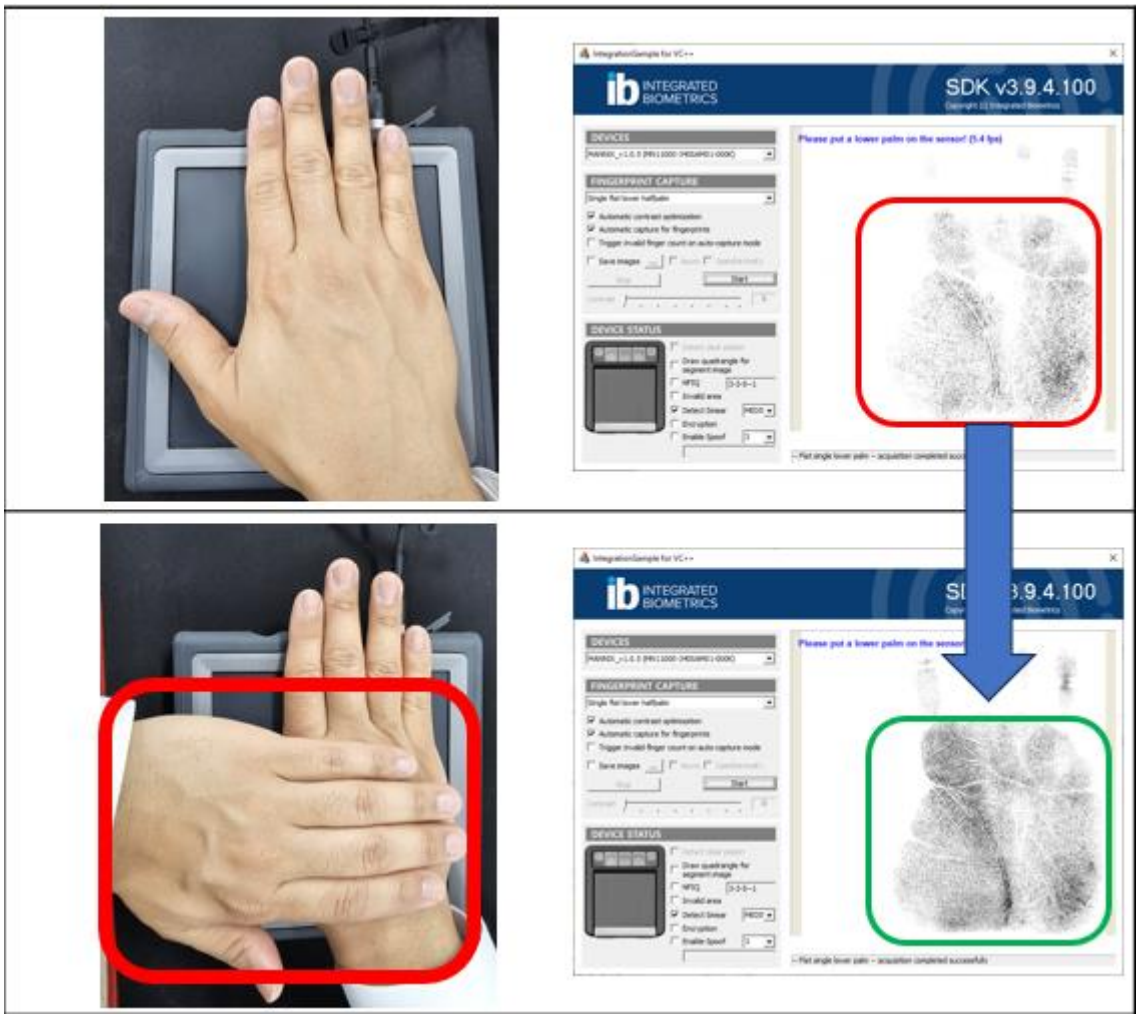
Figure 5. Rolled capture, step 3. The finger should be rolled right until the entire right side is captured

3.3. Mannix product Capture guide

3.3.1. Rolled Print capture

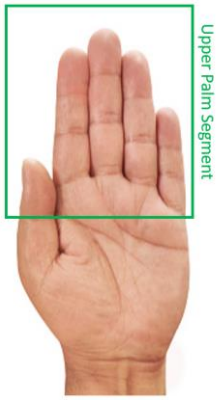
3.3.2. Lower Palm Capture



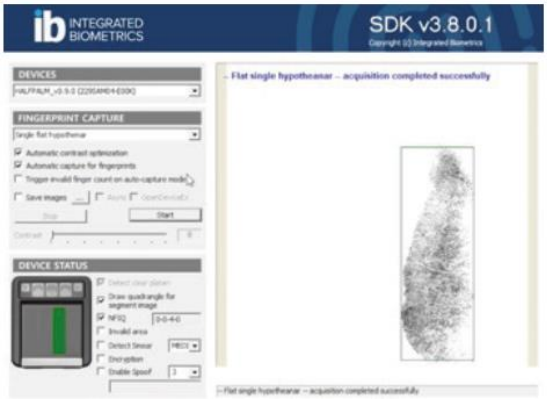


Our Mannix scanner supports “Lower Palm” capture sequence. To improve palm fingerprint recognition, you can either press the hand you want to scan with another hand or ask a colleague next to you to press your hand. This way, you can obtain better palm fingerprints.

3.3.3. Upper Palm Capture



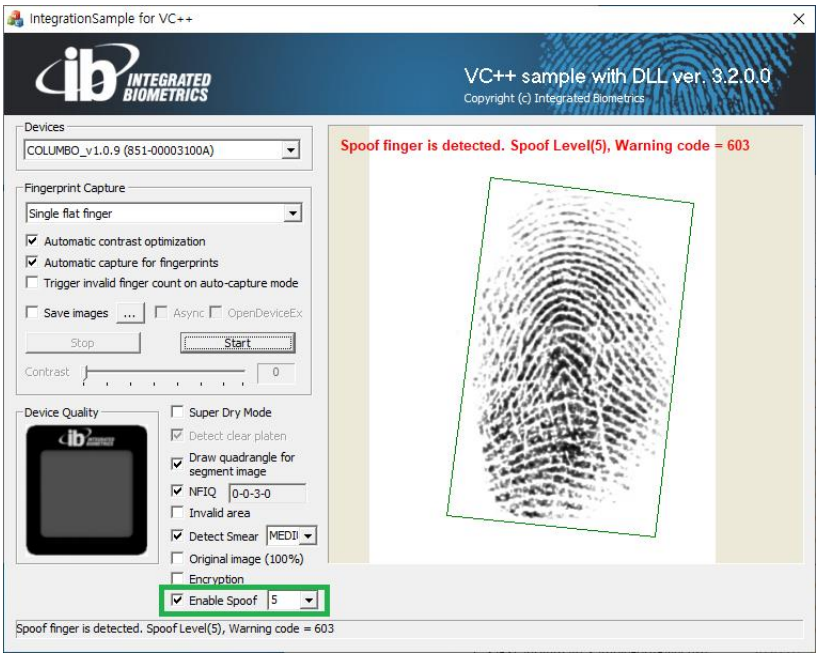
3.3.4. Writer's Palm Capture



Our Mannix scanner supports "Writer's palm" capture sequence.

3.4. Spoof usage

Starting with IBScanUltimate version 3.2.0, Spoof function was enabled.
You can test the Spoof function on our Sample For VC program.
If you want to Enable the Spoof function, check the “Enable Spoof” checkbox and select a Threshold value for Spoof threshold. The default Spoof On/Off property is “Off” and the Threshold property is “5”



4. ISO Template support

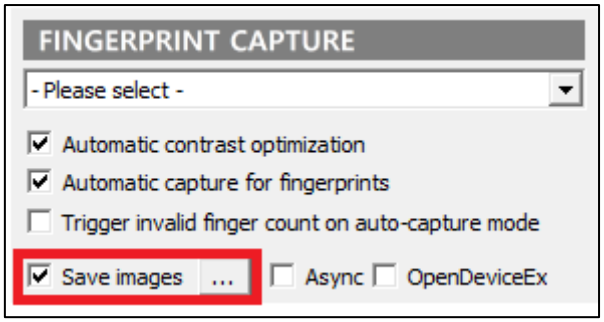
4.1. Introduce ISO Template

Starting with IBScanUltimate version 3.9.0, Our Fingerprint image can convert to an ISO Template.
Our IBScanUltimate supports the following ISO standards.

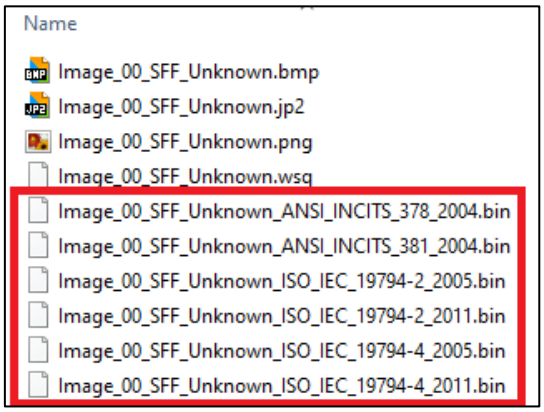
- ISO_19794_2_2005
- ISO_19794_4_2005
- ISO_19794_2_2011
- ISO_19794_4_2011
- ANSI_INCITS_378_2004
- ANSI_INCITS_381_2004

4.2. Fingerprint image Export to ISO Template

Starting with IBScanUltimate 3.9.0 SDK,
When you check the “Save images” option on any of our Sample apps,



The fingerprint images save as ISO template files as shown below.



For detailed explanation of the ISO template conversion, please look into our IBScanUltimate's
C API - IBSU_ConvertImageToISOANSI()
and
Java API - ConvertImageToISOANSI()

4.3. Verify ISO template with BiOCTS

Once you convert Fingerprint images to an ISO template, the ISO template files should be verified they are working as expected.

NIST(https://www.nist.gov/) provides a BioCTS for ISO/IEC program which verifies if the ISO template has the correct standard data.

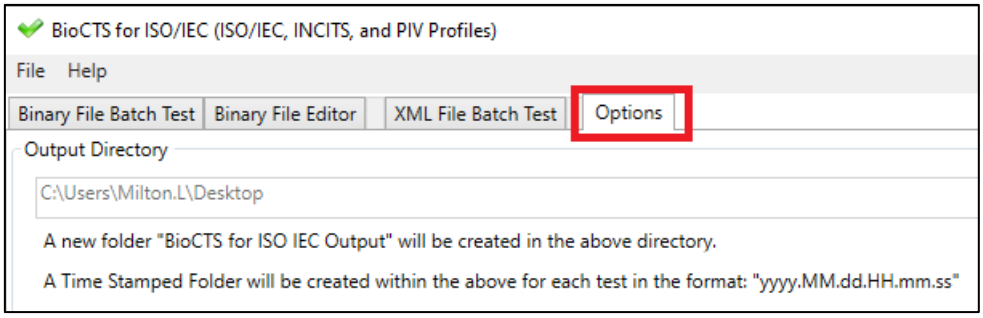
You can download the BioCTS for ISO/IEC program from the link below.

[Link for BioCTS for ISO/IEC]

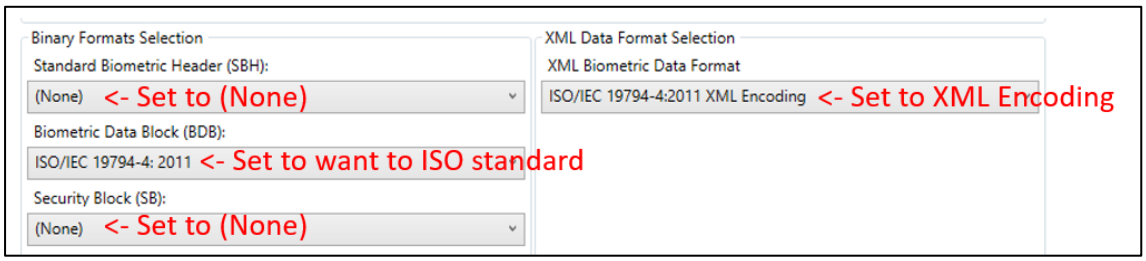
https://www.nist.gov/itl/csd/biometrics-resource-center/biometric-conformance-test-software-biocts/biocts-isoiec

Once the download is completed, Install the BioCTS program on your computer.

After installing, Run the BioCTS program and go to the "Options" tab



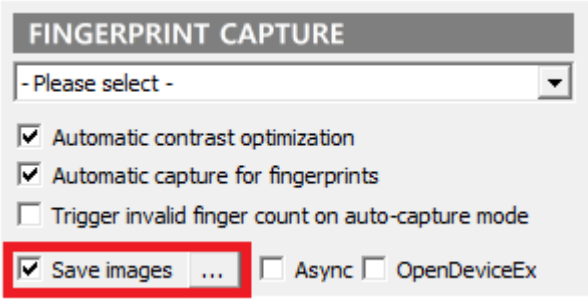
Change the settings as shown below.



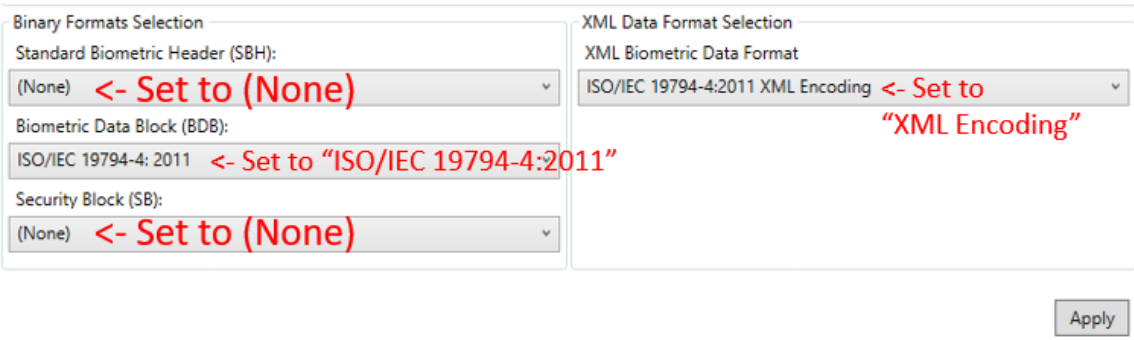
Standard Biometrics Header(SBH) : Set to (None)
Biometrics Data Block(BDB) : Select which ISO/ANSI standard you want to test
Security Block(SB) : Set to (None)
XML Biometric Data Format : Set to XML Encoding

- **Test Example** (Verify **ISO 19794-4:2011 standard)**

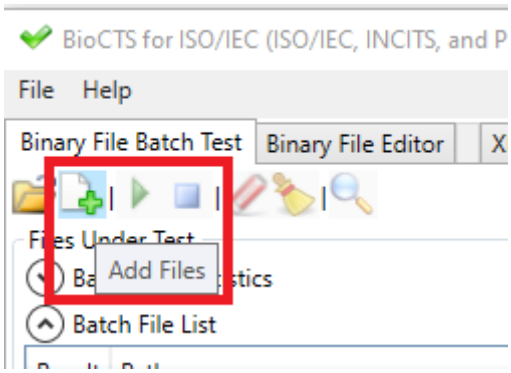
1) Capture Fingerprint image with the "**Save image**" option



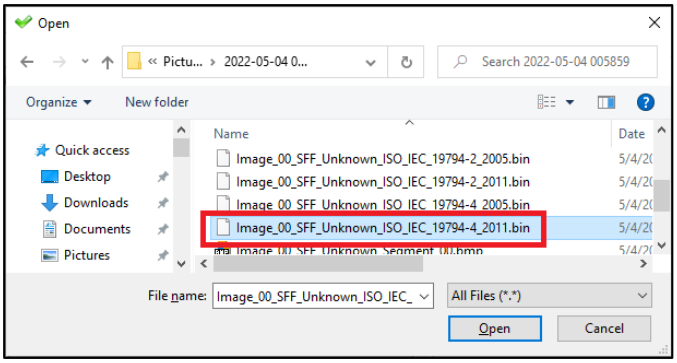
2) Run BioCTS program, and then go to "**Options**" tab and change "Biometrics Data block" value change to "**ISO 19794-4:2011**". Click "Apply" button on the Bottom-Right



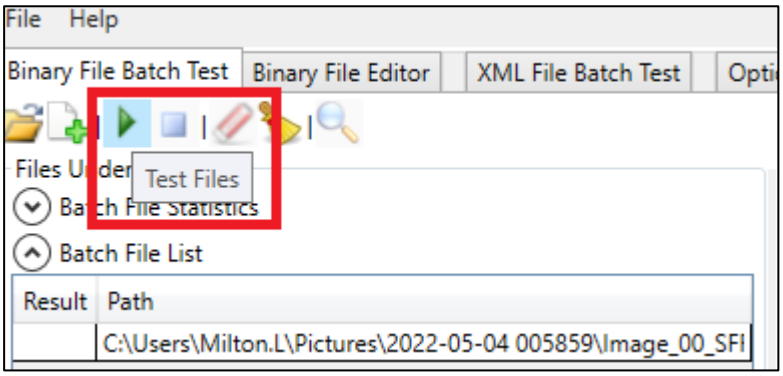
3) Go to the "Binary File Batch Test" tab and click the "Add Files" button



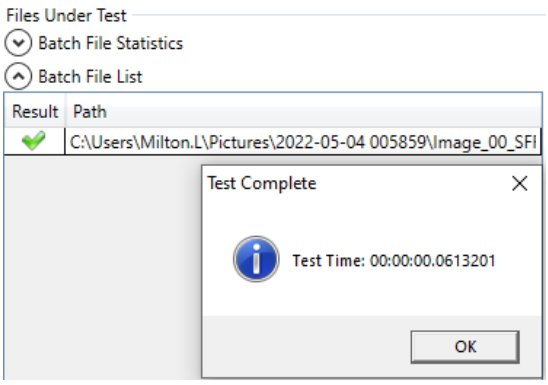
4) Select ISO-19794-4:2011 file in ISO File Saved Location



5) Click the “Test Files” Play button



6) Awesome! Your Standard template is perfect!
(Result field has a Green-Check if the test Passed)



Support Contact Information:

www.integratedbiometrics.com

Integrated Biometrics, LLC

North American Office

Physical Address for Package Delivery
121 Broadcast Drive
Spartanburg SC 29303

For Mailings & Correspondence
PO Box 170938
Spartanburg, SC 29301

US & Canada
(864) 990-3711
Toll-free (888) 840-8034
Extension 1 – Company Directory
Extension 2 – Technical Support
Extension 3 – Sales Support
Extension 4 – Marketing
Extension 5 – Accounting
Extension 0 – Main Line

Sales & Pricing Inquiries
sales@integratedbiometrics.com

Terms & Conditions of a Sale

Terms & Conditions for Supplier Purchases

Sales Administration
marci.bowers@integratedbiometrics.com

Technical Support
technical@integratedbiometrics.com

South Korean Office

Physical Address and Mailing Address
#910 Suntech-City1, 513-15
Sangdaewon 1-dong Jungwon-gu
Seongnam-si, Gyeonggi-do
Republic of Korea

Phone
+82-31-777-2207

Sales Administration
everun@ibkr.co.kr